Beesa Vaishnavi
700772902

Natural Language processing
Homework - 4

Q6a: Multi Input Feedforward:-

Given $x = [2, 1, 3, 1]$

$$A = \begin{bmatrix} 0.2 & -0.5 \\ 0.1 & 0.3 \\ -0.2 & 0.8 \\ 0.4 & -0.6 \end{bmatrix}, \quad B = \begin{bmatrix} 0.7 \\ -1.2 \\ 0.5 \end{bmatrix}$$

(a) Hidden layer

*step ①:- Pre-activations

$Z_1 = 2(0.2) + 1(0.1) + 3(-0.2) + 1(0.4)$

$= 0.4 + 0.1 - 0.6 + 0.4$

$= \boxed{0.3}$

$Z_2 = 2(-0.5) + 1(0.3) + 3(0.8) + 1(-0.6)$

$= -1 + 0.3 + 2.4 + (-0.6) = \boxed{1.1}$

* step ②:- Activations (Sigmoid)

$h_1 = \sigma(0.3) \approx \boxed{0.5744}$

$h_2 = \sigma(1.1) \approx \boxed{0.7503}$

(b) Output Layer

Input to o/p layer : $h = [0.5744, 0.7503, 1]$

step ①: Pre-activation

$Z^{(o)} = 0.5744(0.7) + 0.7503(-1.2) + 1(0.5)$

$= 0.4021 - 0.9004 + 0.5 = \boxed{0.0017}$

step ②:- Output Activation

$y = \sigma(0.0017) \approx \boxed{0.5004}$

(c) Binary Cross - Entropy Loss

Since $t = 1$:

$L = -\log(y)$

$L = -\log(0.5004) \approx \boxed{0.6927}$

Q7 Sol^n: Given network

$h_1 = ReLU(x_1 + x_2)$, $h_2 = ReLU(x_1 + x_2 - 1)$,

$h_3 = ReLU(2x_1 - x_2)$, $y = ReLU(h_1 - 2h_2 + h_3)$

Step (a): Compute o/p for all inputs

We evaluate for all XOR inputs:

Case (1): (0,0)

$h_1 = ReLU(0+0) = 0$

$h_2 = ReLU(0+0-1) = 0$

$h_3 = ReLU(0-0) = 0$

$y = ReLU(0-0+0) = 0$

o/p: 0

Case 2: (0,1)

$h_1 = ReLU(1) = 1$

$h_2 = ReLU(0) = 0$

$h_3 = ReLU(0-1) = 0$

$y = ReLU(1-0+0) = 1$

o/p: 1

Case 3: (1,0)

$h_1 = ReLU(1) = 1$

$h_2 = ReLU(0) = 0$

$h_3 = ReLU(2-0) = 2$

$y = ReLU(1-0+2) = 3$

o/p: 3

Case 4: (1,1)

$h_1 = ReLU(2) = 2$

$h_2 = ReLU(1) = 1$

$h_3 = ReLU(2-1) = 1$

$y = ReLU(2 - 2 + 1) = 1$

o/p = 1

(b) Decision Boundary Comparision

Original XOR network:

$y = ReLU(h_1 - 2h_2)$

outputs:

- $(0,0) \to 0$
- $(0,1) \to 1$
- $(1,0) \to 1$
- $(1,1) \to 0$

perfect XOR behavior (only middle points active)

Extended network:

- Adds extra activation (h_3)
- Increases o/p for $(1,0)$ strongly
- Also activates $(1,1)$, which should be 0 in XOR.

Decision boundary is distorted/shifted.

(c) Does it still compute XOR?

No, it doest compute XOR exactly.

Why?

Correct XOR:

- $(1,1) \to 0$ x but here $\to 1$
- $(1,0) \to 1$ but here $\to 3$

Extra unit $h_3 = ReLU(2x_1 - x_2)$ activates for:

- $(1,0) \to$ large value
- $(1,1) \to$ also positive

This breaks XOR separation.

final o/p is
- network o/p : [0, 1, 3, 1]
- Decision boundary is no longer XOR-like
- fails for input (1,1).

Q.8 sol^n Given

Decision rule: $y = \begin{cases} 1 & \text{if } w_1x_1 + w_2x_2 + b > 0 \\ 0 & \text{otherwise} \end{cases}$

Parameters:

$w_1 = 1, w_2 = -2, b = 1$

Dataset
- $(2,1) \to 1$
- $(1,3) \to 0$
- $(3,2) \to 1$
- $(0,1) \to 0$

(a) Decision Boundary

Decision boundary occurs when:

$w_1x_1 + w_2x_2 + b = 0$

$x_1 - 2x_2 + 1 = 0$

Rewriting:

$x_2 = \frac{x_2 + 1}{2}$ This is a straight line with slope $= 1/2$.

(b) classification of each point

we compute : $f(x) = x_1 - 2x_2 + 1$

* $(2,1) \Rightarrow 2 - 2(1) + 1 = 1 > 0 \Rightarrow y = 1$

correct

* $(1,3) \Rightarrow 1 - 6 + 1 = -4 \Rightarrow y = 0$

correct.

$*(3,2) \Rightarrow 3-4+1=0 \Rightarrow y=0$
misclassified (should be 1)
$*(0,1) \Rightarrow 0-2+1=-1 \Rightarrow y=0$
Correct
Misclassified Points
only: $(3,2)$
(c) Perceptron Loss
Loss = number of misclassified points
Loss = 1
(d) weight update
Perceptron update rule:

$$w = w + \eta(t-y)x$$
$$b = b + \eta(t-y)$$

for misclassified point $(3,2)$:
- True label $t=1$
- Predicted $y=0$
- Learning rate $\eta = 0.5$

update weights: $w_1 = 1 + 0.5(1)(3) = 2.5$
$w_2 = -2 + 0.5(1)(2) = -1$
update bias: $b = 1 + 0.5(1) = 1.5$